

Systematic Taxonomy and Architectural Decision Framework for LLM-Driven Game Systems

Fabian Keßler

Seminar Games Engineering

Julius-Maximilians-Universität Würzburg

Würzburg, Germany

fabian.kessler@stud-mail.uni-wuerzburg.de

Abstract—While Large Language Models (LLMs) have enabled novel dynamic gaming experiences, developers currently lack systematic architectural guidance for LLM integration decisions. This paper addresses this gap through systematic comparative analysis of five seminal LLM-game implementations spanning diverse applications. Through detailed examination of these systems, we identify three fundamental architectural dimensions that characterize current integration approaches: Generation Timing (development-time vs runtime), Integration Depth (wrapper vs embedded systems), and Prompt Architecture (single vs batched prompts). Our analysis reveals how these dimensions interact to influence system complexity, performance characteristics, and development requirements. Based on this systematic examination, we propose an architectural taxonomy and decision framework that maps integration approaches to specific use cases and project constraints. We further analyze recurring implementation patterns across architectures, including prompting strategies, context management techniques, error handling approaches, and input/output processing methods. The framework is grounded in existing implementations and provides structured guidance for developers across different project scales. Finally, we examine how recent LLM advances in multimodal processing, extended context capabilities, and performance optimization may reshape future architectural possibilities, suggesting potential extensions to our current three-dimensional framework.

Index Terms—Large Language Models, Game Architecture, Integration Framework

I. INTRODUCTION

While Procedural Content Generation (PCG) has been a key component in game development for decades [1], the rise of Large Language Models (LLMs) has opened up new possibilities for creating dynamic, player-responsive gaming experiences.

This paper addresses the lack of systematic guidance for selecting appropriate LLM integration architectures. Existing implementations range from lightweight wrapper systems that interface LLMs with existing game engines to comprehensive embedded systems that transform entire content creation workflows. While each approach demonstrates success within its specific domain, the absence of a unified architectural taxonomy prevents developers from making informed design decisions based on their particular requirements and constraints. Based on a systematic analysis of five seminal works spanning diverse gaming applications—from runtime behavior generation to narrative scene creation—we identify

three fundamental architectural dimensions that characterize LLM integration approaches: Generation Timing (devtime vs runtime), Integration Depth (wrapper vs embedded), and Prompt Architecture (single vs batched prompts). Each dimension represents critical design decisions that significantly impact system behavior, development complexity, and player experience. We systematically analyze how these dimensions manifest across different implementations and their associated trade-offs in terms of latency, control, and architectural complexity.

The remainder of this paper is structured as follows. Section II provides background and related work context. Section III details our systematic methodology for analyzing LLM integration architectures. Section IV presents our architectural taxonomy organized around three fundamental dimensions. Section V examines cross-architectural implementation patterns including prompting strategies, context management, error handling, and input/output processing. Section VI introduces an architectural decision framework with practical use case analysis. Section VII discusses how LLM evolution impacts future architectural possibilities. Section VIII presents broader implications and study limitations. Section IX concludes with directions for future research.

II. BACKGROUND & RELATED WORK

The emergence of Large Language Models has fundamentally transformed interactive game system possibilities. Brown et al.’s GPT-3 demonstrated few-shot learning capabilities that enable dynamic adaptation to new game contexts without retraining [2], while Park et al.’s Generative Agents showed how LLMs could create believable autonomous characters with memory and social dynamics [3]. Chen et al.’s work on code generation established LLMs’ ability to synthesize behavior programmatically [4], enabling runtime content creation previously impossible with traditional rule-based systems. These foundational capabilities have enabled sophisticated game applications ranging from dynamic NPC dialogue systems to autonomous multi-agent coordination platforms.

Recent work has explored diverse LLM-game integration approaches but lacks systematic architectural guidance. Volum et al. demonstrated runtime NPC behavior generation in Minecraft through API wrapper architectures [10], while Kumaran et al. developed development-time narrative scene

generation pipelines [5]. Gong et al. implemented centralized multi-agent coordination systems [6], Peng et al. created dynamic text adventure frameworks [7], and Jennings et al. explored runtime behavior compilation in Unity [8]. Despite these advances, existing work focuses primarily on domain-specific implementations rather than systematic architectural guidance. While Gallotta et al.’s comprehensive survey establishes a taxonomy of LLM roles in games [9], no framework exists for guiding architectural decisions across different integration approaches. This paper addresses this gap by developing a systematic taxonomy and decision framework for LLM-driven dynamic game system architectures.

III. METHODOLOGY

This systematic analysis examines five seminal works in LLM-driven game systems through a structured comparative framework designed to identify architectural patterns and technical approaches. The selected papers span diverse gaming applications: Minecraft NPC generation [10], interactive narrative scene creation [5], multi-agent coordination [6], text-adventure emergence [7], and runtime behavior generation [8]. These papers were chosen because of their wide range of use cases, technical feasibility demonstrated through integration with established platforms (Unity, Minecraft, TextWorld), and comprehensive implementation details that enable systematic architectural analysis.

Each paper underwent systematic decomposition using a standardized analysis framework examining five key dimensions: architectural type, game domain/application, technical approach, performance metrics, and explicitly stated limitations. This framework enables direct comparison across different implementation strategies while preserving domain-specific insights.

By examining how each paper actually implements their LLM integration, three fundamental architectural dimensions emerge that characterize different design decisions. The first dimension concerns *generation timing*: whether LLM processing occurs during development time or runtime. Kumaran et al. demonstrate devtime generation through their multi-stage pipeline architecture, while Volum et al., Peng et al., and Jennings et al. implement runtime generation where LLMs respond dynamically during gameplay [5], [7], [8], [10]. The second dimension addresses *integration depth*: whether LLMs function as wrapper layers around existing systems or are embedded into core game architecture. Wrapper approaches are exemplified by Volum et al.’s syntactic sugaring approach with the Mineflayer API and Peng et al.’s implementation where “core logic implemented using TextWorld...employ GPT-4, allowing the game input and output to be dynamic” [7], [10]. Embedded approaches are demonstrated by Jennings et al.’s runtime system where “GROMIT generates behaviors by making requests to a LLM for program code which it then compiles and runs” [8]. The third dimension considers *prompt architecture*: whether systems use individual prompts for discrete processing units (entities, tasks, or events) or batch multiple units into single prompts, with Gong et al.

implementing batched prompt coordination, noting that “while other works...simulate each agent individually, we employ a centralized system.” [6].

The analysis prioritized identifying gaps and limitations explicitly acknowledged by the original authors, as these represent the most reliable indicators of current architectural constraints. This approach ensures our architectural decision framework addresses real implementation challenges rather than theoretical concerns.

IV. ARCHITECTURAL TAXONOMY

The systematic analysis reveals three fundamental architectural dimensions that characterize LLM integration approaches in dynamic game systems. Each dimension represents a critical design decision that significantly impacts system behavior, development complexity, and player experience.

A. Generation Timing: Devtime vs Runtime

The temporal dimension of LLM integration represents a fundamental architectural choice that cascades through all aspects of system design, significantly impacting both player experience and system complexity. This dimension encompasses two primary approaches: development-time generation and runtime generation.

Development-time (Devtime) Generation occurs during the content creation phase, before player interaction. Kumaran et al. exemplify this approach with SceneCraft’s multi-stage pipeline that transforms “high-level author descriptions into playable episodes” through structured workflows [5]. Their architecture processes author intent workbooks through interactive dialogue generation, asset mapping, and script compilation, producing finalized narrative scenes before deployment. This approach offers predictable output quality, reduced runtime computational overhead, and enhanced developer control over final content.

Runtime Generation processes LLM requests dynamically during active gameplay, responding to player actions in real-time. Volum et al. demonstrate this with their Minecraft system, enabling NPCs to generate contextual responses during player interaction [10]. Similarly, Jennings et al.’s GROMIT system generates behaviors by “making requests to a LLM for program code which it then compiles and runs” during gameplay [8]. Peng et al. implement runtime generation in their text adventure where the “core logic implemented using TextWorld...employ GPT-4, allowing the game input and output to be dynamic” [7].

Runtime generation enables highly dynamic player responsiveness and emergent content creation but introduces significant challenges. Peng et al. report “noticeable 15s lag between entering text and receiving response” while Jennings et al. document average latency of 10.6 seconds [7], [8]. Additionally, runtime systems face quality control challenges, with Jennings et al. achieving only 54% overall success rate for generated behaviors [8].

B. Integration Depth: Wrapper vs Embedded

The integration depth dimension characterizes how deeply LLMs are integrated into existing game architectures, ranging from minimal-modification wrapper approaches to comprehensive embedded systems that transform core game workflows.

Wrapper Architectures position LLMs as interface layers around existing game systems, minimizing modifications to established codebases. Volum et al. demonstrate this approach by using Minecraft’s Mineflayer (JavaScript) API with “*syntactic sugaring*” to make the API more accessible to LLM code generation [10]. Their system maintains the original game logic while adding an LLM-compatible interface layer. Similarly, Peng et al. implement a wrapper approach where “*core logic implemented using TextWorld...employ GPT-4, allowing the game input and output to be dynamic*” without modifying TextWorld’s fundamental architecture [7].

Wrapper architectures offer rapid prototyping capabilities and reduced development risk, as they preserve existing game functionality while adding AI capabilities. However, they inherit limitations from underlying systems and may face constraints from API dependencies. Volum et al. acknowledge that their approach suffers from “*calling non-existent functions*” when the LLM generates code that exceeds the available API’s functionality [10].

Embedded Architectures integrate LLMs directly into core game systems, fundamentally altering content creation and interaction workflows. Jennings et al.’s GROMIT system exemplifies this approach through “*runtime compilation*” where the system continuously compiles and executes generated C# code, re-prompting the LLM when errors occur until working code is produced [8]. Their architecture transforms Unity’s standard development workflow by enabling real-time behavior synthesis during gameplay.

Embedded systems enable deeper AI integration and more sophisticated content generation but require comprehensive system redesign. Jennings et al. note that “*programs designed without GROMIT in mind are unlikely to work well with it,*” highlighting the architectural commitment required [8]. While embedded approaches achieve higher success rates in optimized environments (85% in designed scenes vs 30% in unoptimized scenes), they face significant complexity in error handling, security concerns, and developer control trade-offs.

C. Prompt Architecture: Single vs Batched Prompts

The prompt architecture dimension addresses how LLM systems structure their API calls, specifically distinguishing between individual prompts per processing unit versus batching multiple units into single prompts. This architectural choice fundamentally affects coordination complexity, computational efficiency, and emergent behavior patterns.

Single Prompt Systems make individual API calls to the LLM for each processing unit, handling one unit at a time through separate requests. Volum et al. design their system around individual NPC conversations within Minecraft, with each NPC conversation handled through separate API calls while maintaining conversational history until token limits are

reached [10]. Kumaran et al.’s SceneCraft generates individual narrative scenes through separate LLM requests [5]. Peng et al. manage player-world interactions within their text adventure framework, while Jennings et al.’s GROMIT processes individual behavior generation requests with minimal context carryover for error correction [7], [8].

Single prompt approaches offer predictable computational costs per processing unit, simplified error isolation, and straightforward debugging processes. However, they incur higher API call overhead when managing multiple units and may miss opportunities for complex emergent behaviors that arise from coordinated processing across multiple units.

Batched Prompt Systems batch multiple processing units into single prompts, allowing the LLM to process and coordinate several units within a single API call. Gong et al. implement such a batched prompt architecture through their centralized coordination system, noting that “*while other works...simulate each agent individually, we employ a centralized system*” [6]. Their MindAgent architecture manages 2-4 agents simultaneously through a single LLM dispatcher with environmental feedback and Chain-of-Thought reasoning, processing all agents within single prompts rather than separate API calls.

The prompt batching approach offers significant advantages in API call efficiency and coordination quality. Gong et al. report that their architecture “*significantly reduces the number of API calls*” compared to individual agent simulation while enabling coordination through global state awareness within single prompts [6]. Their system measures collaboration effectiveness through a “*Collaboration Score*” metric, which quantifies the proportion of successfully completed tasks relative to all task attempts across varying time constraints [6]. However, this evaluation focuses on varying agent numbers and task difficulty rather than comparing batched versus individual prompt architectures.

Batched prompt systems introduce substantial complexity challenges. Gong et al. note that “*the system performance degrades with more agents under less demanding conditions*” [6]. The batched architecture creates single points of failure and scaling challenges as processing unit counts increase. Additionally, batched prompt systems require sophisticated prompt engineering to maintain distinct processing unit characteristics (e.g. character personalities) while coordinating complex interactions within single prompts.

V. CROSS-ARCHITECTURAL IMPLEMENTATION PATTERNS

All five papers analyzed implemented at least one of these cross-architectural patterns to improve performance in specific technical domains, though the depth and sophistication of implementation varied significantly across different pattern categories. While every system addressed fundamental input/output processing and error handling requirements, more advanced techniques like sophisticated prompting strategies and context management were selectively adopted based on each architecture’s specific operational constraints and performance requirements.

A. Prompting Strategies

The analyzed systems reveal a notable divide in prompting sophistication, with few-shot prompting implemented in three systems (Volum et al. [10], Gong et al. [6], and Jennings et al. [8]), while Peng et al. relies on basic zero-shot approaches [7]. Jennings et al. note that *"few-shot examples can be provided to demonstrate intended output. Indeed, we used both these methods in GROMIT to achieve basic functionality"* [8]. Advanced techniques like Chain-of-Thought reasoning remain underutilized despite Gong et al. demonstrating that they *"can significantly boost multi-agent planning performance by leveraging an emergent in-context learning ability...explaining the rationale of certain actions as in Chain-of-Thought prompting"* [6].

Zero-shot Classification SceneCraft implements zero-shot classification for asset library mapping, using LLMs *"as semantic encoders in a zero-shot classification to map the emotes and gestures for each scene utterance to the most suitable option in our asset library"* with neutral state defaults when uncertain [5].

Few-shot Prompting provides example demonstrations within prompts to guide LLM behavior patterns. Both Volum et al. and Gong et al. implement few-shot prompting. Volum et al. use *"a small number of sample instances in the prompt"* within their seed prompt, then append conversation history where *"the prompt includes the seed prompt plus the previous player commands and model completions"* [10]. Gong et al. demonstrate that *"adding only a few expert demonstrations (from different games) to the prompt"* enables cross-domain coordination without fine-tuning [6].

Chain-of-Thought Reasoning guides LLMs through step-by-step reasoning processes to improve decision quality. Only Gong et al. implement this technique, *"explaining the rationale of certain actions as in Chain-of-Thought prompting"* to enhance multi-agent coordination effectiveness [6].

B. Context Management

LLM token limits pose fundamental constraints for dynamic game systems, requiring sophisticated context management strategies to maintain coherent interactions across extended gameplay sessions. As Gong et al. observe: *"Due to context length limits, it is infeasible to fit the entire history into the context window"* [6], necessitating strategic approaches to context preservation. The following analysis adopts Weber’s prompt component taxonomy, distinguishing between static **Instructions**, dynamic **State** information, and specific **Task** definitions to characterize different context management approaches [11].

History Truncation/Reset represents the simplest approach, where systems revert to initial prompts when limits are exceeded. Volum et al. implement this strategy: *"When the prompt exceeds the allowed token limit (2048 tokens), we revert to the seed prompt and report to the player that the context has been reset"* [10]. While computationally efficient and straightforward to implement, this approach sacrifices contextual continuity (Figure 1).

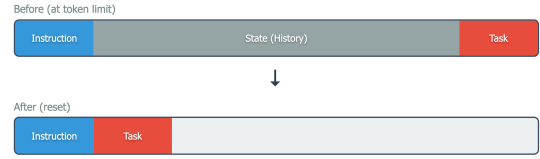


Fig. 1. History Truncation/Reset: Complete context removal when token limits are reached.

History Summarization preserves essential context through intelligent compression. Peng et al. demonstrate this approach: *"When we approach GPT-4’s prompt token limit, we summarize the game history"* [7]. This technique maintains narrative coherence while managing memory constraints, though it requires an additional LLM call to generate summaries, introducing computational overhead (Figure 2).

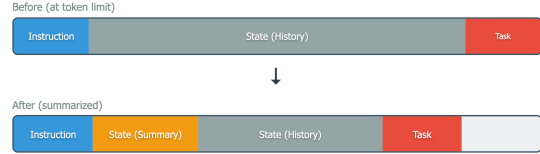


Fig. 2. History Summarization: Compressing older context while preserving recent interactions.

Fixed Horizon History maintains a sliding window of recent interactions. Gong et al. implement this strategy by selecting a fixed horizon history as part of the prompt [6]. This approach balances memory efficiency with temporal relevance (Figure 3).

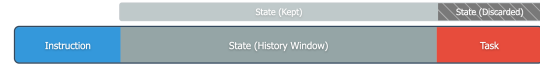


Fig. 3. Fixed Horizon History: Maintaining a sliding window of recent context.

Multi-stage Prompting decomposes complex queries into sequential sub-tasks, where each stage builds upon previous results. Liu et al. demonstrate this technique in dialogue systems: first generating relevant knowledge, then producing responses based on both context and generated knowledge [12]. However, more advanced implementations can employ parallel divide-and-conquer strategies, where complex queries exceeding context limits are split into parallel sub-processes before recombination. Zhang et al. show that such divide-and-conquer prompting substantially improves LLM performance on tasks with repetitive sub-components [13]. While powerful, this approach requires extensive developer preparation to define precise decomposition strategies (Figure 4).

Each technique presents distinct trade-offs between computational efficiency, context preservation, and implementation complexity, with current systems favoring simpler approaches despite their limitations in maintaining long-term narrative coherence.

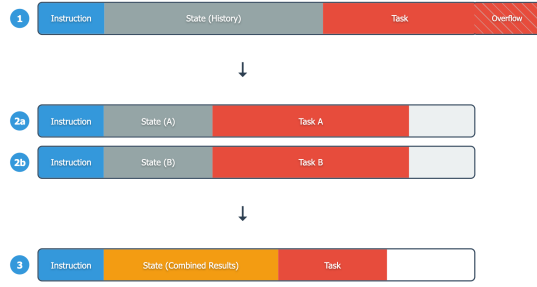


Fig. 4. Multi-stage Prompting: Breaking complex queries into parallel sub-processes.

C. Error Handling

LLM-driven game systems require sophisticated error handling strategies to address the inherent unpredictability of generated content, as LLMs can generate syntactically correct but functionally invalid outputs, fail to follow API constraints, or produce content that breaks game mechanics. The analyzed systems reveal four primary error handling approaches, ranging from reactive correction to proactive prevention, each targeting different types of system failures and operational requirements.

Re-prompting and Iterative Correction represents the most common approach across multiple systems. Kumaran et al. implement this for Python object generation: *“As the output from the LLM is stochastic, it may sometimes be necessary to run these requests to the LLM multiple times to generate a valid Python script object”* [5]. Jennings et al. demonstrate similar techniques for C# compilation errors through *“re-prompting the system with the error information”*, contributing to their overall 54% system success rate [8]. While computationally expensive, this approach enables recovery from generation failures without manual intervention.

Action Validation and Look-ahead Checking provides proactive error prevention through pre-execution validation. Gong et al. implement comprehensive validation: *“Action Validation utilizes a look-ahead checking mechanism...If an action is deemed unexecutable, an error message is returned”* [6]. Their system achieves effective error prevention because it operates within a constrained action space of five simple operations, enabling reliable conflict detection between agents attempting incompatible actions.

Enhanced Failure Messaging transforms technical errors into contextually appropriate game responses. Peng et al. demonstrate this approach where the system *“either executes the action or returns a predefined failure message. To make failure messages more palatable, GPT-4 generates either an NPC response...or a game feedback message”* [7]. This technique successfully maintains player immersion while handling system limitations, though no quantitative metrics are provided for player satisfaction with failure responses.

Automatic Correction and Runtime Isolation addresses common LLM generation errors through architectural solutions. Jennings et al. implement *“compilation tricks”* to handle

“forgetting import statements and forgetting class wrappers” [8]. Their architecture achieves complete crash prevention through component isolation: *“No behavior generated for either demo resulted in the game crashing in any way. This is largely due to GROMIT handling compilation errors internally, and runtime errors being limited in scope”* [8].

The comparative analysis reveals a systematic trade-off between error handling sophistication and implementation complexity. Reactive approaches (re-prompting, enhanced messaging) offer broad applicability but incur computational overhead and latency costs. Proactive approaches (action validation, automatic correction) achieve higher reliability but require constrained environments or extensive architectural redesign. However, validation coverage remains inconsistent across systems. While Gong et al. implement sophisticated action validation within their constrained domain [6], other systems exhibit significant gaps—Volum et al. acknowledge that their NPC *“may attempt to call a function that does not exist in the API”* without implementing prevention mechanisms [10], and Jennings et al. rely primarily on post-compilation error correction rather than input validation [8]. This uneven implementation reflects the substantial architectural investment required for comprehensive validation, with research prototypes often prioritizing core functionality demonstration over robust error prevention mechanisms.

D. Input/Output Processing

LLM-driven game systems require sophisticated input/output processing mechanisms to bridge the gap between natural human interaction patterns and structured game system requirements. The analyzed systems reveal distinct approaches to handling diverse input modalities and transforming LLM outputs into executable game actions.

Input Classification represents a preprocessing step where systems categorize player inputs to determine appropriate processing pathways. Peng et al. implement an explicit classification where *“GPT-4 first determines whether the player is trying to perform an in-game action ([action]) or to interact with an NPC ([words])”* before routing to corresponding system components [7].

Structured Input Transformation converts complex inputs into LLM-compatible formats through systematic decomposition. Kumaran et al. demonstrate this with an *“Author intent workbook”* that captures narrative requirements in predefined categories before LLM processing [5]. This approach addresses the challenge of translating *“high-level author descriptions into playable episodes within a 3D virtual world”* by systematically decomposing authorial intent into four structured elements: character background, scene location, interaction scenario, and conversation tone [5].

Environmental State Representation addresses the challenge of conveying 3D game worlds to text-based LLMs. Jennings et al. develop Spatial Scene Graphs that *“can be filtered to only include nodes related to the input request”* to manage context complexity and enable selective environmental awareness [8].

Multi-modal Input Integration extends beyond text, incorporating other types of inputs such as audio and gesture inputs. Jennings et al. implement spoken audio processing through Whisper audio-to-text models, which *"can be supplemented with pointing gestures"* using raycasting to determine object selection [8]. This approach addresses accessibility and immersion but introduces additional processing latency and error propagation from multiple input streams.

Output Parsing and Execution transforms LLM responses into game-executable actions through structured extraction mechanisms. Gong et al.'s action extraction *"employs a regular expression matching procedure to distill agent actions from the textual output"* [6]. Jennings et al. implement runtime compilation with automatic correction for common generation errors like missing imports and class wrappers before code execution [8].

VI. DECISION FRAMEWORK AND USE CASE ANALYSIS

Based on our architectural taxonomy of Generation Timing, Integration Depth, and Prompt Architecture, we present a systematic decision framework for selecting optimal LLM integration approaches in dynamic game systems.

A. Architectural Decision Matrix

Our framework provides actionable guidance through a comprehensive decision matrix that maps architectural combinations to optimal use cases, implementation complexity, and key technical considerations. Each architectural combination presents distinct trade-offs between development efficiency, runtime performance, and system flexibility, enabling developers to select approaches aligned with their specific constraints and requirements.

B. Framework Application and Complexity Assessment

Implementation complexity is determined by our three taxonomic dimensions: Runtime generation increases complexity through real-time performance requirements and latency management; Embedded integration increases complexity requiring architectural redesign and deep system modifications; Batched prompting increases complexity through sophisticated coordination logic and failure propagation across processing units.

Applying this framework to our analyzed systems: Volum et al.'s Minecraft NPC system and Peng et al.'s text adventure framework (Runtime + Wrapper + Single) represent low complexity through preserved existing architectures with isolated processing [7], [10]. Kumaran et al.'s narrative scene generation pipeline (Devtime + Embedded + Single) achieves medium complexity via offline processing but requires custom toolchain integration [5]. Jennings et al.'s Unity runtime behavior compilation (Runtime + Embedded + Single) demonstrates high complexity through real-time architectural modifications [8]. Gong et al.'s multi-agent coordination system (Runtime + Embedded + Batched) exhibits very high complexity by combining all complexity-increasing factors [6].

External implementations provide preliminary validation: MarioGPT's procedural level generation (Devtime + Embedded + Single) demonstrates medium complexity patterns [14], while StoryVerse's dynamic plot generation (Runtime + Embedded + Single) aligns with high complexity expectations [15].

C. Application Scenarios

Indie RPG Development: Small teams with limited resources and existing Unity codebases requiring dynamic NPC dialogue should select **Runtime + Wrapper + Single** architecture. This approach minimizes system modifications while enabling player-responsive dialogue, as demonstrated by Volum et al.'s successful Mineflayer integration [10]. Low implementation complexity aligns with tight development timelines and budget constraints.

AAA Content Pipeline Integration: Studios requiring consistent, high-quality environmental storytelling assets across multiple projects should implement **Devtime + Embedded + Single** architecture. This combination enables quality control through offline generation, custom toolchain integration, and predictable costs without runtime performance impact. Medium complexity suits dedicated development teams with established infrastructure.

Multi-Agent Research Platforms: Academic teams investigating emergent coordination behaviors require **Runtime + Embedded + Batched** architecture despite very high complexity. Real-time multi-agent coordination necessitates sophisticated prompt batching and deep system integration, as validated by Gong et al.'s MindAgent implementation [6]. Complex architectures enable advanced research capabilities but demand significant development investment.

D. Framework Limitations

The framework assumes discrete architectural choices, but real-world systems often require hybrid approaches combining multiple strategies within single projects. For example, a system might use development-time generation for core narrative content while employing runtime generation for player-specific dialogue responses, blurring the Generation Timing dimension boundaries.

The framework operates at a high level of abstraction, focusing on fundamental architectural dimensions rather than specific implementation details. While this abstraction enables broad applicability across diverse game genres and development contexts, it may not capture the nuanced technical considerations that arise during actual implementation.

Finally, the framework's validation relies on a limited set of research prototypes rather than production game systems. The architectural patterns observed in academic implementations may not fully reflect the constraints and requirements of commercial game development, where factors like development timelines, team expertise, and business model considerations significantly impact architectural choices.

TABLE I
ARCHITECTURAL DECISION MATRIX FOR LLM-GAME INTEGRATION

Architecture	Best For	Example Applications	Key Considerations	Implementation Complexity
Devtime + Wrapper + Single	Rapid prototyping, integration into existing Editors	Character backstories, Item descriptions	High quality control, No ongoing runtime cost	Low
Devtime + Wrapper + Batched	Balanced content sets, Thematic consistency	Weapon stat balancing, Location atmosphere sets	Internal consistency, Batch processing efficiency, Static relationships	Low-Medium
Devtime + Embedded + Single	Custom toolchain integration, Code generation pipelines	Behavior script generation, Custom shader generation	Direct engine access, Compilation overhead, Architectural coupling	Medium
Devtime + Embedded + Batched	Coordinated system generation, Interdependent code modules	Multi-agent behavior systems, Interconnected game mechanic generators	System-wide coherence, Complex dependencies, Extensive architectural redesign	High
Runtime + Wrapper + Single	Individual NPC interactions, Player feedback	Dynamic dialogue, Adaptive hints	Player responsiveness, API limitations, Easier debugging	Low
Runtime + Wrapper + Batched	Multi-entity scenes, Cost optimization	Group conversations, Coordinated dynamic event instantiation	Reduced API calls, Shared context, Failure propagation	Medium-High
Runtime + Embedded + Single	Experimental mechanics, Player-driven content	Procedural spell effects, Dynamic quest generation	Maximum flexibility, Security concerns	High
Runtime + Embedded + Batched	Multi-agent systems, Emergent ecosystems	Team AI collaboration, Dynamic faction behavior	Complex emergence, Exponential complexity, Single point of failure	Very High

VII. LLM EVOLUTION AND FUTURE ARCHITECTURAL IMPLICATIONS

Recent LLM advances directly impact our taxonomic dimensions, suggesting extensions beyond the current framework.

Multimodal Integration and Spatial Understanding addresses the persistent limitation identified across our analyzed systems regarding visual game world comprehension. Current systems rely on textual conversion layers, as Volum et al. acknowledge that *"cases strongly suggest the need for the NPC to have visual capabilities"* [10]. Recent multimodal LLMs demonstrate direct visual processing capabilities, with models achieving between 10% and 25% of human performance in complex visual environments like Atari games while outperforming the random agents, *"demonstrating a meaningful level of understanding and ability to play the games"* [16]. However, *"spatial reasoning remains a significant challenge for multimodal large language models"* [16]. These developments particularly impact Integration Depth decisions, as systems can now process raw visual game states rather than requiring wrapper-based textual abstractions, though spatial reasoning limitations constrain embedded implementation effectiveness.

Extended Context and Long-form Reasoning capabilities in contemporary LLMs directly address context management limitations documented in our analysis. While *"some models like Gemini could even to be capable of dealing with millions of tokens"* [17], recent evaluation frameworks reveal significant performance degradation on complex reasoning tasks. On challenging classification tasks, *"all LLMs achieve close-to-zero performance except Gemini-1.5-Pro with 14% accuracy"* compared to 87% for fine-tuned models [17]. Furthermore, *"long context understanding and reasoning is still a chal-*

lenging task for the existing LLMs" with models exhibiting *"bias towards labels presented later in the sequence"* [17], suggesting that architectural patterns emphasizing context compression and selective attention remain essential.

Performance Optimization and Real-time Viability through advances in serving systems enable Runtime Generation architectures in previously infeasible scenarios. Systems achieving *"up to 2.3x higher effective throughput, 2x lower latency on average, and up to 3.7x lower (token-level) tail latency"* [18] make real-time LLM integration feasible for latency-sensitive gaming applications. As performance improvements reduce computational costs and latency penalties, Runtime Generation becomes increasingly attractive compared to Devtime alternatives.

These technological advances suggest that while our three-dimensional architectural taxonomy remains foundational, future LLM-game architectures will likely require additional dimensions addressing multimodal integration patterns (e.g., NPCs processing both visual player positioning and natural language simultaneously), adaptive context management strategies (e.g., dynamic importance weighting of narrative elements), and hybrid generation timing approaches (e.g., predictive pre-generation with real-time fallbacks) that blur the traditional devtime-runtime distinction.

VIII. DISCUSSION

A. Framework Synthesis and Practical Impact

Our three-dimensional architectural taxonomy provides systematic guidance for LLM-game integration decisions, with Generation Timing emerging as the most fundamental dimension due to its cascading effects on system complexity and player experience. The analysis reveals that Runtime

Generation, while enabling dynamic responsiveness, requires careful consideration of latency constraints and error handling complexity that significantly impact architectural choices in the Integration Depth and Prompt Architecture dimensions.

B. Study Limitations

This analysis focuses on five seminal works representing early LLM-game integration approaches, potentially limiting generalizability to emerging architectural patterns. The rapid evolution of LLM capabilities means that current performance constraints (latency, context limits, success rates) may not accurately reflect future system requirements. Additionally, our framework emphasizes technical architectural dimensions while giving limited attention to game design considerations such as player agency, narrative coherence, and ethical implications of AI-generated content.

C. Broader Implications

The systematic approach demonstrated here extends beyond game development to other interactive AI applications requiring real-time content generation, suggesting broader applicability for human-AI interaction systems in virtual environments, educational platforms, and creative tools.

IX. CONCLUSION

We introduce the first taxonomy for LLM-driven game systems, defined by three key dimensions: Generation Timing, Integration Depth, and Prompt Architecture. Through systematic analysis of five seminal implementations, we demonstrate how these dimensions interact to determine system complexity, performance characteristics, and development requirements.

Our decision framework offers practical guidance for developers selecting appropriate architectural approaches based on specific project constraints and requirements. The validation through existing systems confirms the framework's descriptive accuracy and practical utility, while the cross-architectural analysis reveals common technical challenges and solutions.

As LLMs advance—especially in multimodal processing, context handling, and inference speed—future architectures will likely extend beyond our current model. As demonstrated in our analysis of emerging technologies, next-generation systems will likely integrate visual understanding, adaptive context management, and hybrid generation timing approaches that fundamentally reshape the architectural design space.

Future research should investigate the integration of multimodal capabilities into existing architectural patterns, develop standardized evaluation frameworks for LLM-game system performance, and examine how different architectures affect player experience and game design. Additionally, longitudinal studies of production LLM-game systems will provide valuable insights into the evolution and maturation of these architectural approaches in real-world deployment scenarios.

X. ACKNOWLEDGMENTS

Figures 1–4 were created by the author with assistance from Claude Sonnet 4 and subsequently modified to fit the research requirements. Large Language Models, particularly

Claude Sonnet 4, were used throughout the writing process to support formulation and refinement of ideas, argumentation structure, and technical expression.

REFERENCES

- [1] M. Hendrikx, S. Meijer, J. Van Der Velden, and A. Iosup, "Procedural content generation for games: A survey," *ACM Trans. Multimedia Comput. Commun. Appl.*, vol. 9, no. 1, pp. 1–22, Feb. 2013, doi: 10.1145/2422956.2422957.
- [2] T. B. Brown et al., "Language Models are Few-Shot Learners," July 22, 2020, arXiv: arXiv:2005.14165. doi: 10.48550/arXiv.2005.14165.
- [3] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative Agents: Interactive Simulacra of Human Behavior," Aug. 06, 2023, arXiv: arXiv:2304.03442. doi: 10.48550/arXiv.2304.03442.
- [4] Chen et al., "Evaluating Large Language Models Trained on Code," July 14, 2021, arXiv: arXiv:2107.03374. doi: 10.48550/arXiv.2107.03374.
- [5] V. Kumar, J. Rowe, B. Mott, and J. Lester, "SceneCraft: Automating Interactive Narrative Scene Generation in Digital Games with Large Language Models," *AIIDE*, vol. 19, no. 1, pp. 86–96, Oct. 2023, doi: 10.1609/aiide.v19i1.27504.
- [6] R. Gong et al., "MindAgent: Emergent Gaming Interaction," in *Findings of the Association for Computational Linguistics: NAACL 2024*, Mexico City, Mexico: Association for Computational Linguistics, 2024, pp. 3154–3183. doi: 10.18653/v1/2024.findings-naacl.200.
- [7] X. Peng et al., "Player-Driven Emergence in LLM-Driven Game Narrative," in *2024 IEEE Conference on Games (CoG)*, Milan, Italy: IEEE, Aug. 2024, pp. 1–8. doi: 10.1109/CoG60054.2024.10645607.
- [8] N. Jennings, H. Wang, I. Li, J. Smith, and B. Hartmann, "What's the Game, then? Opportunities and Challenges for Runtime Behavior Generation," in *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*, Pittsburgh PA USA: ACM, Oct. 2024, pp. 1–13. doi: 10.1145/3654777.3676358.
- [9] R. Gallotta et al., "Large Language Models and Games: A Survey and Roadmap," *IEEE Trans. Games*, pp. 1–18, 2024, doi: 10.1109/TG.2024.3461510.
- [10] R. Volum et al., "Craft an Iron Sword: Dynamically Generating Interactive Game Characters by Prompting Large Language Models Tuned on Code," in *Proceedings of the 3rd Wordplay: When Language Meets Games Workshop (Wordplay 2022)*, Seattle, United States: Association for Computational Linguistics, 2022, pp. 25–43. doi: 10.18653/v1/2022.wordplay-1.3.
- [11] I. Weber, "Large Language Models as Software Components: A Taxonomy for LLM-Integrated Applications," June 13, 2024, arXiv: arXiv:2406.10300. doi: 10.48550/arXiv.2406.10300.
- [12] Y. Liu et al., "Agent Design Pattern Catalogue: A Collection of Architectural Patterns for Foundation Model based Agents," Nov. 06, 2024, arXiv: arXiv:2405.10467. doi: 10.48550/arXiv.2405.10467.
- [13] Y. Zhang, L. Du, D. Cao, Q. Fu, and Y. Liu, "An Examination on the Effectiveness of Divide-and-Conquer Prompting in Large Language Models," July 02, 2024, arXiv: arXiv:2402.05359. doi: 10.48550/arXiv.2402.05359.
- [14] S. Sudhakaran, M. González-Duque, C. Glanois, M. Freiberger, E. Najjarro, and S. Risi, "MarioGPT: Open-Ended Text2Level Generation through Large Language Models," Nov. 08, 2023, arXiv: arXiv:2302.05981. doi: 10.48550/arXiv.2302.05981.
- [15] Y. Wang, Q. Zhou, and D. Ledo, "StoryVerse: Towards Co-authoring Dynamic Plot with LLM-based Character Simulation via Narrative Planning," in *Proceedings of the 19th International Conference on the Foundations of Digital Games*, May 2024, pp. 1–4. doi: 10.1145/3649921.3656987.
- [16] N. R. Waytowich, D. White, M. D. Sunbeam, and V. G. Goecks, "AtariGPT: Benchmarking Multimodal Large Language Models as Low-Level Policies in Atari Games," Dec. 02, 2024, arXiv: arXiv:2408.15950. doi: 10.48550/arXiv.2408.15950.
- [17] T. Li, G. Zhang, Q. D. Do, X. Yue, and W. Chen, "Long-context LLMs Struggle with Long In-context Learning," 2024, arXiv. doi: 10.48550/ARXIV.2404.02060.
- [18] C. Holmes et al., "DeepSpeed-FastGen: High-throughput Text Generation for LLMs via MII and DeepSpeed-Inference," Jan. 09, 2024, arXiv: arXiv:2401.08671. doi: 10.48550/arXiv.2401.08671.